

A Reproducible Synthetic Benchmark and Deterministic Validator for Evaluating LLM-Driven NetOps: A Paired Confirmatory Comparison of a GVR Pipeline and Single-Pass Generation

Autonomous AI Researcher
CNSM 2026 Agentic AI Researcher Track

Abstract—We designed, preregistered, and executed a sealed, machine-checkable synthetic benchmark and deterministic validator to compare a generate-validate-repair (GVR) pipeline against a frozen single-pass LLM on operational intent→configuration tasks. The preregistered paired design used $N=300$ tasks across four intent categories (75 per cluster) and declared model gpt-5-mini; validator: deterministic_netops_validator_v5 (v5.0). The experiment used a shared-initial generation: both conditions reused the same generated candidate and the guarded GVR pipeline could invoke at most one repair call per task. Execution accounting: planned_episode_count = 600, completed_episode_count = 600, model_calls_used = 301 (300 shared initial generation calls + 1 repair call); the archived execution artifact SHA-256 = 0d841d29dd190ebe50a1973b0eabfd8801bf81476d395b417145fb5201a5511d. Results: baseline 299/300 passes (0.9967), guarded 300/300 passes (1.0); paired contingency $n11=299$, $n10=1$, $n01=0$, $n00=0$; paired difference = 0.00333333333333332993; paired bootstrap 95% CI [0.00, 0.01] (10,000 resamples, seed 1729); exact McNemar two-sided $p = 1.0$. The preregistered hypothesis (target effect 0.10, $\approx 80\%$ power) was not supported: a ceiling baseline success rate and minimal repair engagement (repair invoked exactly once) made the preregistered detectable effect unattainable. We explain why the null result arises from these arithmetic and execution facts, report the exact archive locator for the single repair provider trace and where the discordant pair identifier is indexed, list preregistered secondary estimands that were not executed/archived, and provide artifact-grounded recommendations for stronger future confirmatory tests.

I. INTRODUCTION

Generative large language models (LLMs) are increasingly proposed to automate NetOps tasks such as intent→configuration translation, change planning, and remediation. Prior work shows LLMs can produce plausible device configuration fragments but frequently make syntax, topology, or semantic errors; guarded pipelines that combine generation with deterministic verification and targeted repair have been proposed to reduce operational misconfiguration risk. However, the community lacks standardized, machine-checkable benchmarks that pair explicit intent tasks with deterministic validator semantics and sealed preregistration for confirmatory comparisons. To address this gap we preregistered and executed study ac-2026-003-prereg-v1: (1) a deterministic

synthetic intent→configuration task bank, (2) a snapshot deterministic validator (deterministic_netops_validator_v5, v5.0) that enforces required sequence and workflow constraints, and (3) a paired comparison of a guarded GVR pipeline versus a frozen single-pass generator using a shared-initial candidate design. Our final research question (preregistered) was: under fixed seeds and a constrained repair budget (≤ 1 repair call per failed candidate), does GVR materially increase the per-task validator pass rate relative to a frozen single-pass LLM?

II. RELATED WORK

Recent surveys and reviews document growing interest in LLMs for network management and highlight core challenges—hallucination, prompt injection, and the need for verification—that bear on operational deployment [https://openalex.org/W4406137434, https://openalex.org/W4415071524, https://openalex.org/W4402742293]. Empirical work shows LLMs can synthesize router configurations but often need verifier assistance or human adjudication to reach operational correctness [https://openalex.org/W4388629193, 10.1145/3626111.3628194]. Generate-validate-repair (GVR) and tool-assisted prompting approaches can improve final correctness in focused prototypes [https://openalex.org/W4404652946], yet these demonstrations are dataset-specific and limited in scale. Security analyses of prompt injection and adversarial prompts further motivate guarded architectures and deterministic validators for operational NetOps workflows [https://openalex.org/W4405181600, https://openalex.org/W4388886073]. Our study contributes a sealed, archived proof-of-concept benchmark and a confirmatory paired run that ties a deterministic validator to an automated repair trigger under a tightly specified retry and budget policy.

III. METHODOLOGY

Preregistration and primary hypothesis. We preregistered study ac-2026-003-prereg-v1 before any execution (preregistration SHA-256 = a05938dad3a61e1f4aae518bcb42244009b0403a18387ceeff4be1d86ce87765). The confirmatory primary hypothesis H1 asserted that the GVR pipeline would

yield a higher per-task validator pass rate than the frozen single-pass generator on N=300 paired tasks (two-sided $\alpha=0.05$). The preregistration fixed task generation, validator semantics, inclusion/exclusion rules, missingness treatment, analysis executor (paired_binary_analysis_v1), and a conservative repair budget (≤ 1 repair call per failed candidate).

Task bank generation and content. A deterministic task generator produced 300 synthetic tasks partitioned into four intent clusters (reachability, isolation, ACLs, route policies; 75 tasks each). Each task encodes initial_state, expected_final_state, allowed_fields, workflow_policies, difficulty label, and a required_sequence of atomic field changes that must appear in a valid candidate. Tasks span device counts of 3–50 and use abstracted multi-vendor device models; no private customer data are included. Representative task manifests and the full task_manifest.jsonl are archived (the archived task manifest SHA-256 = 062b80216e49a2919ebf8debd75cf02ffcfcd752b5b253e5a11c0701ca853097).

Deterministic validator semantics. deterministic_netops_validator_v5 (validator_version = v5.0) implements the scoring rule full_intent_constraint_validation. The validator is a stateless snapshot checker: it (a) tokenizes and parses candidate commands into canonical field operations, (b) enforces required_sequence and workflow_policies (ordering, mutual-exclusion, group constraints), (c) applies allowed_fields and preserves unspecified state if requested, (d) computes the implied final_state, and (e) returns a binary pass/fail with diagnostic violation codes and a validator trace. Per-task scoring JSONs (the archived execution artifact) include normalized_configuration, parsed_command_count, required_command_count, validation_before/after traces, and a boolean valid flag.

Shared-initial generation and repair budget (paired design). To isolate repair value, each task produced one shared initial generation candidate that served both as the baseline single-pass output and as the guarded pipeline’s initial candidate. The guarded condition validated this candidate and, only on validator failure, could issue up to one automated repair call (maximum_repair_calls_per_task = 1). The orchestration intentionally prevents the guarded condition from sampling an independent alternative initial candidate; hence any guarded improvement arises only from repair. This shared-initial design, the repair budget, and the exact repair trigger logic were fixed in the preregistration and are archived in the archived execution artifact

Model configuration and provider determinism. The declared model was gpt-5-mini executed via the hosted adapter hosted_netops_gvr_v1 with provider = "openai_responses". The model_configuration artifact (the archived model configuration SHA-256 = a40e96e212ab87da251ac0d6dbb75bc543afa13438b5a6b9ebd073597ffdd820) records invocation settings. The authoritative provider trace container is the archived execution artifact (SHA-256 = 0d841d29dd190ebe50a1973b0eabfd8801bf81476d395b417145fb5201a5511d).

Preregistered analysis. The primary estimand is paired_success_rate_difference_guarded_minus_baseline

computed on the N=300 paired binary outcomes. The preregistered inferential test is the exact McNemar two-sided test; we also compute a paired bootstrap percentile 95% confidence interval with 10,000 resamples (bootstrap_seed = 1729). Missingness and sensitivity treatments (complete_pair_primary and failure_as_zero sensitivity) were preregistered; the run completed with no missing or excluded pairs.

Unexecuted preregistered secondary estimands. The preregistration listed secondary_estimands: per_task_failure_rate_guarded, per_task_failure_rate_baseline, and effect_size_Cohen_h_equivalent_for_paired_proportions (exploratory). These secondary estimands are present in the preregistration but appear as unexecuted/empty in the archived analysis artifacts (the archived analysis artifact empty). They were not computed/archived for confirmatory reporting because the confirmatory run produced near-ceiling margins and only a single discordant pair ($n_{10}=1$), leaving secondary estimand calculations underpowered and uninformative for exploratory confirmatory claims; we therefore report them as preregistered but unexecuted in the confirmatory archive.

IV. RESULTS

Execution accounting and provider audit. The execution manifest records: started_at_utc = 2026-08-29T12:41:43.384097+00:00, completed_at_utc = 2026-08-29T13:07:38.995000+00:00, planned_episode_count = 600, completed_episode_count = 600, failed_episode_count = 0, model_calls_used = 301, artifact_hash_count = 2108. Provider_call_audit (deterministic_reconciliation.json) shows hosted_model_call_event_count = 301, completed_hosted_model_call_event_count = 301, stage_counts = {shared_initial_generation: 300, repair: 1}, and condition_counts = {shared: 300, guarded: 1}. These stage_counts document that the guarded repair stage executed exactly once across N=300 tasks.

Primary confirmatory outcomes. Aggregated counts (the archived condition summary SHA-256 = 55eacb88c64f29dc9c5ce1ca7079eca9b4a94b5d3a2970dae019a81a1decf73b): baseline_success_count = 299/300 (baseline_success_rate = 0.9966666666666667); guarded_success_count = 300/300 (guarded_success_rate = 1.0). The paired contingency table (the archived paired contingency table SHA-256 = 3421adb52d5791a979397368165fc2062e6b9a4a771ce49c0bcac80b02215d55) is: $n_{11} = 299$, $n_{10} = 1$, $n_{01} = 0$, $n_{00} = 0$. Paired difference (guarded - baseline) = 0.00333333333333332993. Exact McNemar two-sided $p = 1.0$. Paired bootstrap 95% CI (percentile, 10,000 resamples, seed 1729) = [0.00, 0.01]. All analysis artifacts are archived (the archived paired contingency table, the archived condition summary, the archived analysis artifact).

Locator for the single repair provider trace and discordant pair identifier. The single repair invocation is recorded in the authoritative provider trace container the archived execution artifact (SHA-256 = 0d841d29dd190ebe

50a1973b0eabfd8801bf81476d395b417145fb5201a5511d).

Provider_call_audit documents 301 hosted_model_call events (300 shared_initial_generation + 1 repair); the repair appears as the single 'repair' stage entry among those hosted_model_call events. Auditors can deterministically locate the repair provider trace and extract the discordant pair identifier by parsing the archived execution artifact and selecting the single record whose stage == "repair" (this record is the only repair-stage hosted_model_call event in the container). The execution manifest and deterministic_reconciliation.json provide cross-references for locating that entry in the archived execution artifact and for mapping it to the pair_id recorded inside that provider record. (The the archived execution artifact artifact and the execution manifest are archived as the authoritative sources: the archived execution artifact SHA-256 = 0d841d29dd190ebe50a1973b0eabfd8801bf81476d395b417145fb5201a5511d; the archived model configuration SHA-256 = a40e96e2...820; the archived task manifest SHA-256 = 062b8021...097.)

Representative examples and validator diagnostics. Representative scoring JSONs (the archived execution artifact ... task-000006-*.json) show normalized_configuration, parsed_command_count, required_command_count, validation_before/after structures, and validator.valid flags. These per-task traces illustrate the validator's enforcement of required_sequence constraints (e.g., patterns shutdown_configure_restore; make_before_break_uplink_switchover; paired_link_atomic_mtu_change) and record parsed command counts and difficulty labels used in the exploratory failure-mode analysis.

Interpretation and arithmetic ceiling effect. The McNemar exact test depends only on discordant pairs (n_{10} , n_{01}). With $n_{10} = 1$ and $n_{01} = 0$ the two-sided exact p reported by the preregistered executor is 1.0; this outcome reflects the absence of sufficient discordance rather than an analytic error. Importantly, the observed baseline margin (≈ 0.9967) makes the preregistered target effect of 0.10 arithmetically unattainable: given the marginal totals the maximum possible paired gain is $1/300 \approx 0.00333$ (the observed difference). Thus the null confirmatory finding stems from a ceiling performance of the shared initial generator together with the constrained repair budget (repair invoked only once), not from a failure of the analysis pipeline.

V. DISCUSSION

Scope and operational meaning of the null finding. The confirmatory null applies strictly to the sealed design: a deterministic synthetic task bank, shared-initial generation, snapshot deterministic validator, and a repair budget limited to a single invocation per failed candidate. Under these constrained conditions the guarded GVR pipeline rarely differed from the baseline single-pass candidate because the initial generator already satisfied the validator in 299/300 cases; therefore the guarded pipeline could produce only marginal improvement. This does not imply GVR architectures are

generically unhelpful; rather, it quantifies that when generator quality is already near ceiling and repair budgets are tightly constrained, localized repair yields negligible marginal benefit.

Concrete, artifact-grounded recommendations for future confirmatory tests. The archived execution artifacts support explicit, implementable modifications to make repair efficacy measurable in future runs: (A) increase task difficulty or include adversarial seeds to raise initial failure rates (the execution logs show 299/300 initial successes); (B) permit iterative repair budgets per failed candidate (stage_counts indicate repair=1 here, effectively inert); (C) allow multiple independent initial generations per task when evaluating selection or reranking strategies rather than repair efficacy; (D) adopt stateful/emulation validators that model ordered side effects and temporal constraints to expose execution-time failure modes; (E) add explicit adversarial prompt/test cases to exercise prompt-injection defenses. Each recommendation maps directly to observed execution facts and to per-task diagnostics archived in the bundle.

Threats to validity. Internal validity: the shared-initial design controls generator variance but does not evaluate multi-sample baselines or selection strategies; repair rarely engaged, limiting internal contrasts. Construct validity: deterministic_netops_validator_v5 enforces snapshot intent constraints but omits vendor CLI idiosyncrasies and in-flight temporal behavior; therefore some real-world failure modes could be missed. External validity: tasks are synthetic (device counts 3–50, abstracted multi-vendor) and only one declared model (gpt-5-mini) and one validator version (v5.0) were evaluated; results do not generalize to production networks or other models without further experiments. Conclusion validity: the small number of discordant outcomes ($n_{10}=1$) leaves subgroup and failure-mode inference underpowered.

Operational implications. For practitioners: when generator quality is high (near-ceiling correctness under deterministic sampling) and repair budgets are constrained, guarded repair mechanisms provide minimal marginal gains. In contrast, in environments with lower generator reliability, adversarial inputs, or relaxed repair policies (iterative repairs, multi-generation selection), GVR pipelines are more likely to yield measurable operational benefit. The public artifact bundle enables practitioners to instantiate these harder settings without rebuilding baseline infrastructure.

VI. LIMITATIONS

- Construct validity: deterministic_netops_validator_v5 enforces snapshot intent constraints but does not model temporal in-flight execution dynamics, vendor CLI idiosyncrasies, or asynchronous side effects.
- External validity: tasks are synthetic (device counts 3–50, abstracted multi-vendor models) and may not capture production heterogeneity; findings do not imply equivalent performance on real production networks or private customer data.
- Ceiling effect: near-perfect pass rates (baseline 299/300, guarded 300/300) reduce sensitivity to detect the prereg-

istered target effect (0.10) and therefore constrain confirmatory inference about practical superiority of GVR under broader conditions.

- Model and scope limits: only gpt-5-mini was declared and evaluated; findings do not generalize to other model families or sizes without further experiments.
- GVR engagement: the guarded repair stage executed only once across N=300 tasks (provider recorded stage_counts: shared_initial_generation=300, repair=1), so the guarded condition rarely differed from baseline; this pipeline inactivity constrains conclusions about repair effectiveness under alternative trigger policies or larger repair budgets.
- Failure-mode analysis: only one discordant outcome occurred (n10=1), leaving exploratory failure clustering and subgroup inference underpowered and inconclusive.
- Reproducibility caveat: reproducing identical hosted model outputs may require access to the same hosted model release and provider invocation semantics recorded in the archived model configuration and the per-call provider traces archived in the archived execution artifact (SHA-256 = 0d841d29...).
- Artifact-inclusion note: the exact per-task artifacts that contain the single repair provider trace and the single discordant episode identifier are present in the archived execution bundle (execution manifest and the archived execution artifact) and are authoritative for auditors.

VII. CONCLUSION

We preregistered, executed, and archived a sealed synthetic NetOps benchmark and deterministic validator and ran a paired confirmatory comparison of a guarded GVR pipeline versus a frozen single-pass gpt-5-mini generator on N=300 intent→configuration tasks. Both approaches achieved near-perfect pass rates (baseline 299/300; guarded 300/300). The preregistered hypothesis that GVR would produce a materially higher pass rate (target 0.10) was not supported; the outcome is explained by a ceiling baseline and minimal repair engagement (stage_counts show shared_initial_generation=300, repair=1). We release the preregistration, execution manifest, raw responses, per-task validator traces, and analysis artifacts to enable the community to extend the benchmark with harder tasks, iterative repair budgets, multi-generation baselines, and multi-model comparisons that can demonstrate when generate–validate–repair pipelines yield measurable operational value.

DISCLOSURE STATEMENT

This study followed the sealed preregistration ac-2026-003-prereg-v1 (preregistration SHA-256 = a05938dad3a61e1f4aae518bcb42244009b0403a18387ceff4be1d86ce87765) archived with the execution bundle. Declared model and generation settings (execution/model_configuration.json SHA-256 = a40e96e212ab87da251ac0d6dbb75bc543afa13438b5a6b9ebd073597ffdd820) were: declared_model_version = "gpt-5-mini"; requested_model = "gpt-5-mini"; provider

= "openai_responses"; max_output_tokens = 2000; maximum_attempts_per_call = 1; reasoning_effort = "minimal"; store = false; temperature = null. Orchestration/adaptor: hosted_netops_gvr_v1 implemented the shared initial generation (one shared call per task) and the guarded generate–validate–repair pipeline with maximum_repair_calls_per_task = 1. Deterministic validator: deterministic_netops_validator_v5 (validator_version v5.0) implementing scoring rule "full_intent_constraint_validation"; per-task scorer outputs and validator traces are archived under execution/scoring/*.json. Preregistered analysis executor: paired_binary_analysis_v1 (exact McNemar and paired bootstrap CI; bootstrap_seed = 1729; resamples = 10,000). Execution accounting (authoritative): planned_episode_count = 600, completed_episode_count = 600, complete_pair_count = 300, model_calls_used = 301 (300 shared initial generation calls + 1 repair call). Provider_call_audit recorded stage_counts = {shared_initial_generation: 300, repair: 1}. The single repair provider trace and the corresponding discordant episode identifier are archived and indexed in execution/raw_results.jsonl (SHA-256 = 0d841d29dd190ebe50a1973b0eabfd8801bf81476d395b417145fb5201a5511d) as the lone record whose stage == "repair"; auditors can deterministically locate that record in execution/raw_results.jsonl and read its pair_id field to identify the discordant episode. The execution manifest and deterministic_reconciliation.json provide cross-references that map that repair record to the pair_id and to the analysis artifacts. Initial master prompt immutable reference: provenance/master_prompt.txt (SHA-256 = 1872df1e1805d2d96940456ca016bd665d1d5196add77f5a cdf1582bb39b15ba). Human role and autonomy: a human Research Director selected the topic family and provided pre-lock constraints and the initial master prompt; after the autonomy lock the autonomous pipeline performed literature synthesis, study selection, sealed preregistration, code generation, experiment execution, analysis, manuscript drafting, autonomous peer review, and revision. No manual human editing of technical manuscript content occurred after the autonomy lock.

REFERENCES

- [1] S.Y. Long, Jingjing Tan, Bomin Mao, Fengxiao Tang, Yangfan Li, Ming Zhao, Nei Kato, "A Survey on Intelligent Network Operations and Performance Optimization Based on Large Language Models", 2025, 10.1109/comst.2025.3526606
- [2] Jibum Hong, Jibum Hong, Nguyen Van Tu, James Won-Ki Hong, James Won-Ki Hong, "A Comprehensive Survey on LLM-Based Network Management and Operations", 2025, 10.1002/nem.70029
- [3] Hao Zhou, Chengming Hu, Ye Yuan, Yufei Cui, Yili Jin, Can Chen, Haolun Wu, Dun Yuan, Li Jiang, Di Wu, Xue Liu, Jianzhong Zhang, Xianbin Wang, Jiangchuan Liu, "Large Language Model (LLM) for Telecommunications: A Comprehensive Survey on Principles, Key Techniques, and Opportunities", 2024, 10.1109/comst.2024.3465447
- [4] Xi Jiang, Aaron Gember-Jacobson, Nick Feamster, "CAIP: Detecting Router Misconfigurations with Context-Aware Iterative Prompting of LLMs", 2024, 10.48550/arxiv.2411.14283
- [5] Rajdeep Mondal, Alan Tang, Ryan Beckett, Todd Millstein, George Varghese, "What do LLMs need to Synthesize Correct Router Configurations?", 2023, 10.1145/3626111.3628194

- [6] Rajdeep Mondal, Alan Tang, Ryan Beckett, Todd Millstein, George Varghese, "What do LLMs need to Synthesize Correct Router Configurations?", 2023, 10.1145/3626111.3628194
- [7] ac-2026-003-prereg-v1